Predicción de popularidad de canciones

Integrantes:
Xavier Alexander Bravo Berroterán
Carlos Alberto Mancilla Valle
Esteban Vallejo Galeano
Materia: Data Analytics

1. Introducción

El presente informe documenta el miniproyecto «Predicción de popularidad de canciones», desarrollado en la materia Data Analytics. Su propósito es estimar, antes del lanzamiento de una canción, la popularidad que alcanzaría en una escala de 0 a 100, de modo que la disquera cuente con un criterio para decidir cuánto invertir en promoción (Figura 13). Para ello se entrenó un modelo de Machine Learning con Python, se expuso mediante un servicio web construido con FastAPI y se desplegó en Render. El contenido se elabora exclusivamente a partir de las capturas de pantalla aportadas como evidencia.

2. Desarrollo técnico

Tecnologías. Las evidencias muestran el uso de Python 3.11.9, pandas, NumPy, scikit-learn y joblib para el análisis y el modelo; FastAPI, Pydantic y Uvicorn para el servicio; pytest para las pruebas; Jupyter y un editor de código para el desarrollo; y Render para el despliegue (Figuras 2, 3, 5, 10, 11 y 13).

Datos y preparación. El conjunto de datos es el archivo canciones.csv, con 2000 registros (Figuras 6 y 7). Se eliminaron las variables reproducciones_sem1 y es_hit por no estar disponibles antes del lanzamiento (Figura 7) y se creó la variable derivada duracion_c2 (Figura 8). La partición fue temporal: canciones de 1995 a 2019 para entrenamiento (1585 registros) y de 2020 a 2025 para prueba (415 registros) (Figuras 8 y 13). La ficha del servicio advierte que los datos son sintéticos y que el modelo describe asociaciones, no relaciones causales (Figura 13).

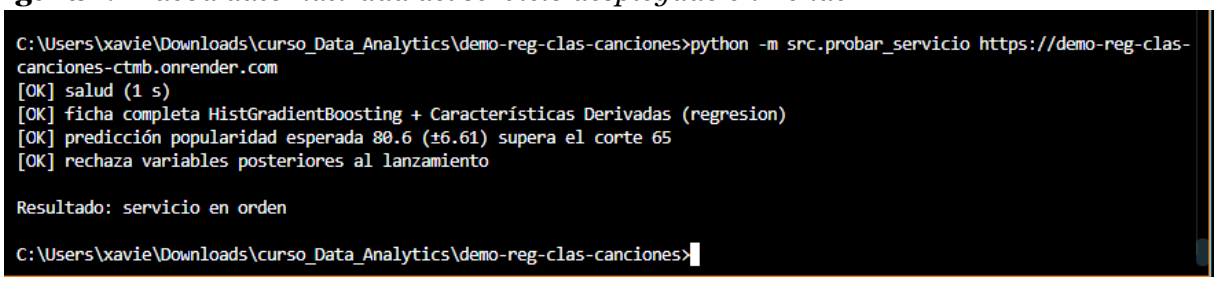
Modelo. Según la ficha publicada por el servicio, el modelo desplegado es un HistGradientBoosting con características derivadas para una tarea de regresión, con MAE de 6,61, RMSE de 8,35 y R^2 de 0,616 en el periodo de prueba, frente a un MAE de 10,44 de la línea base (media por género) (Figura 13). El cuaderno evalúa un pipeline con MAE, RMSE y R^2 (Figura 9) y exporta el modelo, su ficha y las columnas de entrada a models/modelo.pkl mediante joblib (Figura 10).

Servicio y despliegue. El servicio expone las rutas GET /salud, GET /modelo, POST /predecir y POST /predecir_lote, esta última para hasta 1000 canciones (Figuras 12, 15 y 16). El esquema de entrada valida el rango de cada variable y rechaza campos adicionales con error 422 (Figura 17). La respuesta incluye la popularidad esperada, un rango, el corte de promoción y la decisión de promocionar con su explicación (Figuras 14 y 19). El servicio se publicó en Render con despliegue automático (Figura 3) y las versiones de las dependencias se fijan en requirements.txt (Figura 11).

3. Evidencias

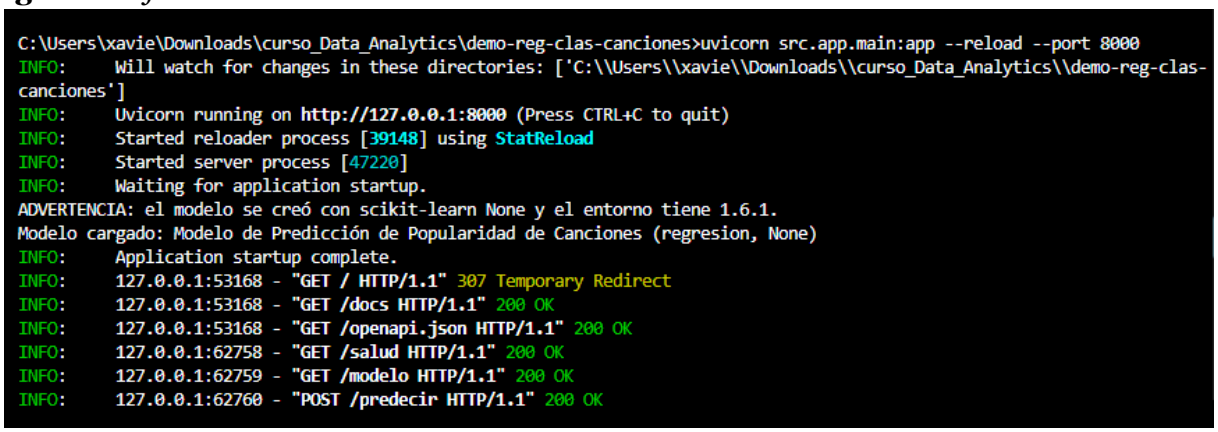
A continuación se presentan las capturas en el mismo orden en que aparecen en el archivo original.

Figura 1. *Prueba automatizada del servicio desplegado en Render*



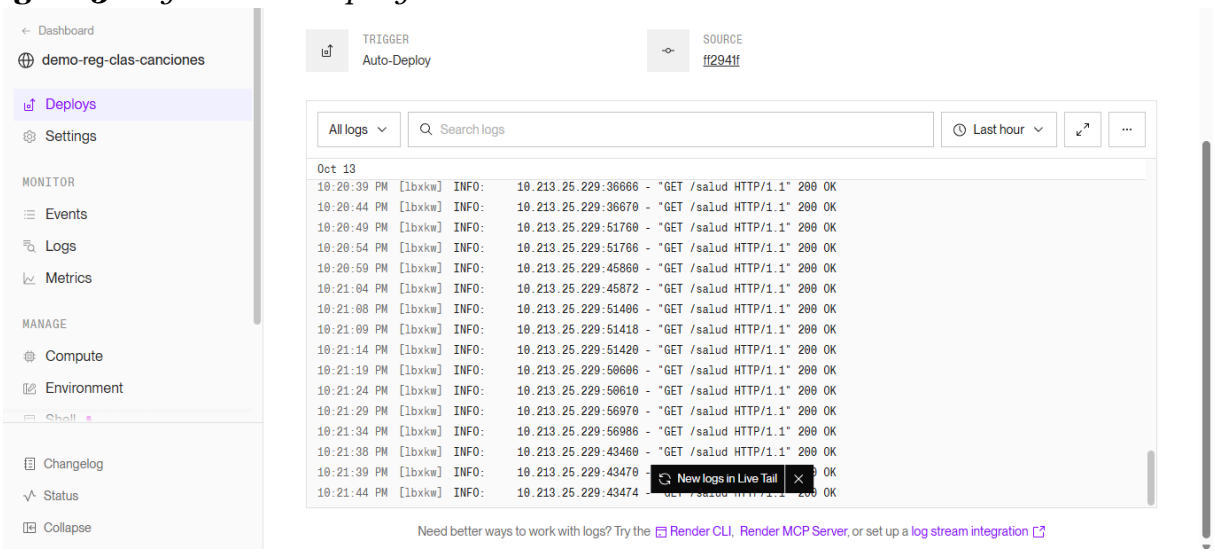
Nota. Salida de la herramienta src.probar_servicio ejecutada contra la URL pública del servicio en Render. Muestra cuatro verificaciones con resultado [OK]: estado de salud (1 s), ficha del modelo HistGradientBoosting con características derivadas (regresión), una predicción de popularidad esperada de 80,6 (±6,61) que supera el corte de 65, y el rechazo de variables posteriores al lanzamiento. La última línea indica «Resultado: servicio en orden».

Figura 2. *Ejecución local del servicio con Uvicorn*



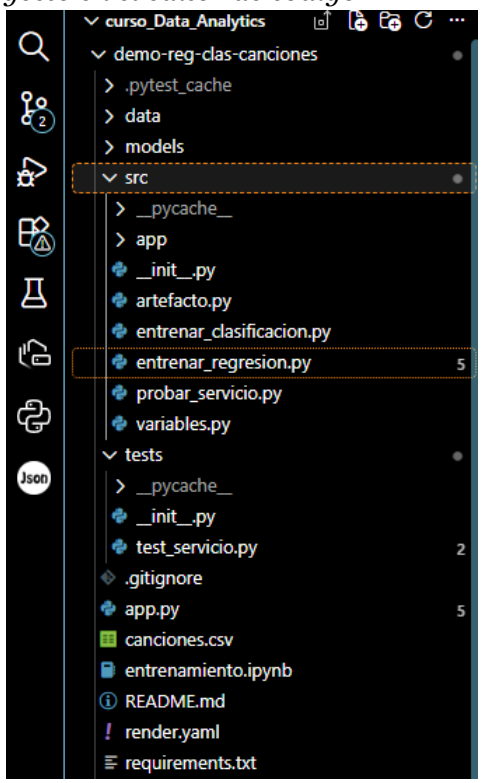
Nota. Registro de consola del comando uvicorn src.app.main:app --reload --port 8000, con el servicio escuchando en http://127.0.0.1:8000. Se observa el mensaje «Modelo cargado: Modelo de Predicción de Popularidad de Canciones (regresion, None)», una advertencia por la diferencia entre la versión de scikit-learn registrada en el modelo (None) y la del entorno (1.6.1), y solicitudes con respuesta 200 OK a /docs, /openapi.json, /salud, /modelo y POST /predecir. Verifica que la aplicación arranca y responde en el entorno local.

Figura 3. *Registros del despliegue en Render*



Nota. Panel de Render del servicio demo-reg-clas-canciones, en la sección de registros, con disparador Auto-Deploy y commit de origen ff2941f. Los registros del 13 de octubre (entre las 10:20 y las 10:21 p. m.) muestran solicitudes GET /salud con respuesta 200 OK a intervalos cercanos a cinco segundos, lo que evidencia que el servicio publicado está activo y responde en ese endpoint.

Figura 4. *Estructura del proyecto en el editor de código*



Nota. Explorador de archivos del proyecto demo-reg-clas-canciones. Se identifican las carpetas data, models, src (con app, artefacto.py, entrenar_clasificacion.py, entrenar_regresion.py, probar_servicio.py y variables.py) y tests (con test_servicio.py), además de los archivos app.py, canciones.csv, entrenamiento.ipynb, README.md, render.yaml, requirements.txt y .gitignore. Muestra la organización del código de entrenamiento, del servicio, de las pruebas y de la configuración de despliegue.

Figura 5. *Importación de librerías en el cuaderno de entrenamiento*

```
# Celda 1: Importar librerías necesarias
import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import TimeSeriesSplit
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

print("Librerías cargadas correctamente.")
```

[11]

```
Librerías cargadas correctamente.
```

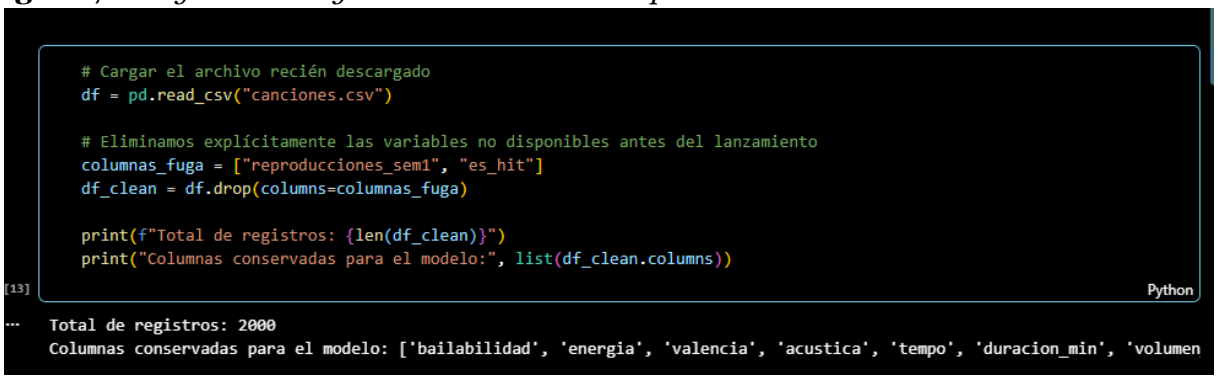
Nota. Primera celda del cuaderno entrenamiento.ipynb. Importa pandas, numpy y joblib, y de scikit-learn TimeSeriesSplit, ColumnTransformer, StandardScaler, OneHotEncoder, Pipeline, Ridge y las métricas mean_absolute_error, root_mean_squared_error y r2_score. La salida «Librerías cargadas correctamente.» confirma que el entorno dispone de las dependencias necesarias.

Figura 6. *Descarga del conjunto de datos canciones.csv*



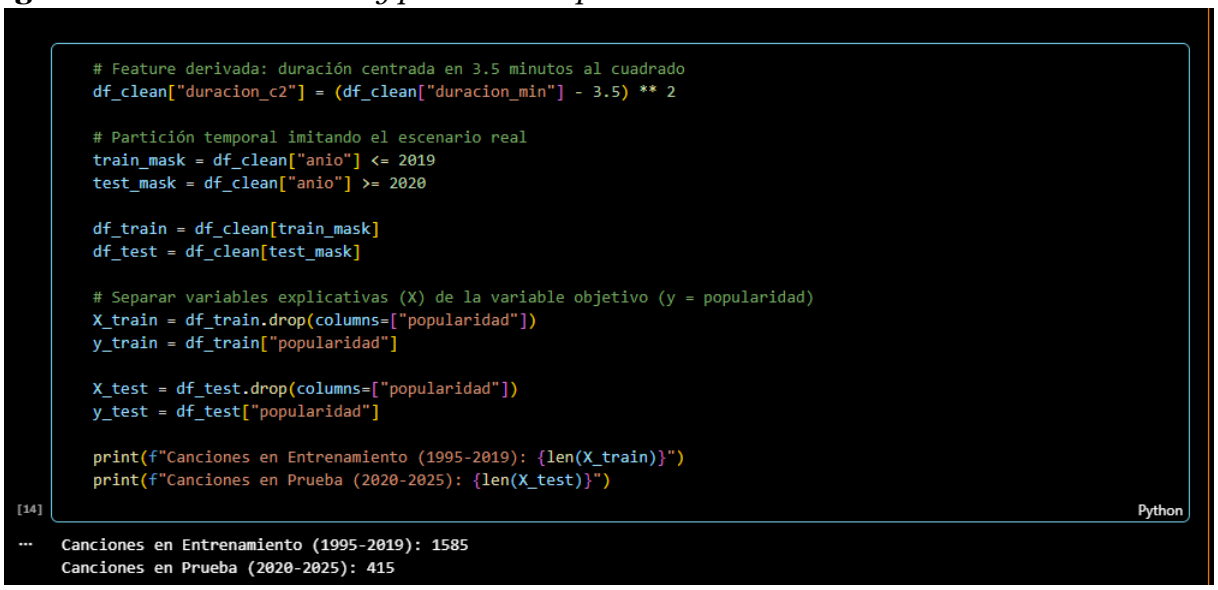
Nota. Celda que descarga el archivo canciones.csv desde un repositorio de GitHub mediante urllib y lo guarda en la carpeta actual. La salida «¡Archivo canciones.csv descargado con éxito!» confirma la obtención del conjunto de datos utilizado en el entrenamiento.

Figura 7. *Carga de datos y exclusión de variables posteriores al lanzamiento*



Nota. Celda que lee canciones.csv y elimina las columnas reproducciones_sem1 y es_hit, descritas en el código como no disponibles antes del lanzamiento. La salida indica un total de 2000 registros y muestra las columnas conservadas (bailabilidad, energia, valencia, acustica, tempo, duracion_min y volumen…; la línea aparece cortada en la captura).

Figura 8. *Variable derivada y partición temporal de los datos*




```
# Feature derivada: duración centrada en 3.5 minutos al cuadrado
df_clean["duracion_c2"] = (df_clean["duracion_min"] - 3.5) ** 2

# Partición temporal imitando el escenario real
train_mask = df_clean["anio"] <= 2019
test_mask = df_clean["anio"] >= 2020

df_train = df_clean[train_mask]
df_test = df_clean[test_mask]

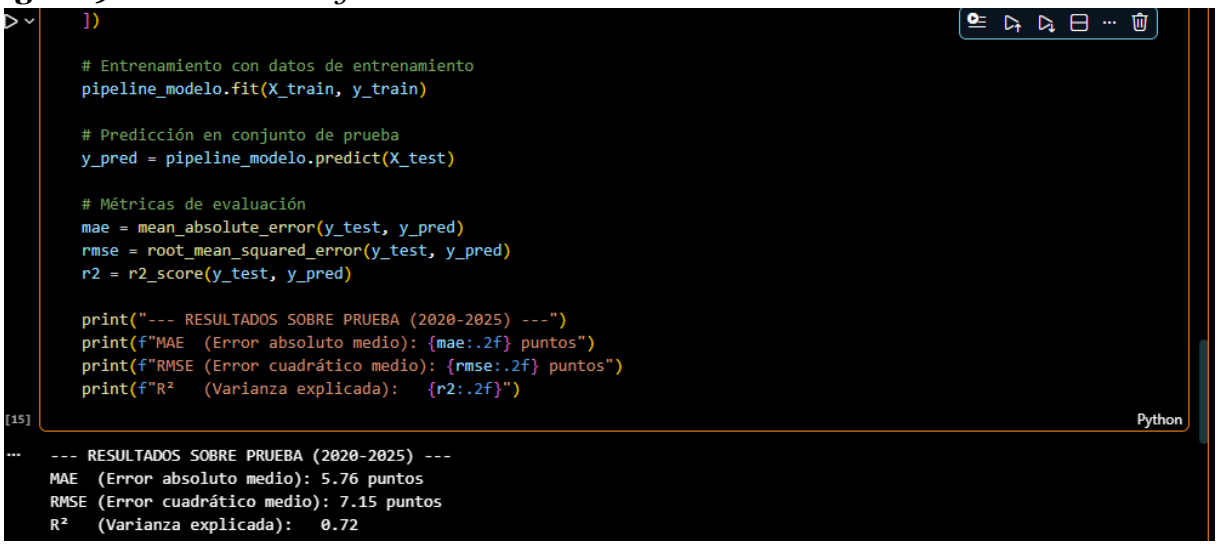
# Separar variables explicativas (X) de la variable objetivo (y = popularidad)
X_train = df_train.drop(columns=["popularidad"])
y_train = df_train["popularidad"]

X_test = df_test.drop(columns=["popularidad"])
y_test = df_test["popularidad"]

print(f"Canciones en Entrenamiento (1995-2019): {len(X_train)}")
print(f"Canciones en Prueba (2020-2025): {len(X_test)}")
```

Nota. Se crea la variable derivada duracion_c2, que corresponde a la duración centrada en 3,5 minutos y elevada al cuadrado. Luego se separan los datos por año: entrenamiento con canciones hasta 2019 (1585 registros) y prueba desde 2020 (415 registros), con popularidad como variable objetivo. La partición temporal simula el escenario real de predicción sobre canciones posteriores a las usadas para entrenar.

Figura 9. *Entrenamiento y evaluación del modelo en el cuaderno*




```
])

# Entrenamiento con datos de entrenamiento
pipeline_modelo.fit(X_train, y_train)

# Predicción en conjunto de prueba
y_pred = pipeline_modelo.predict(X_test)

# Métricas de evaluación
mae = mean_absolute_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("--- RESULTADOS SOBRE PRUEBA (2020-2025) ---")
print(f"MAE  (Error absoluto medio): {mae:.2f} puntos")
print(f"RMSE (Error cuadrático medio): {rmse:.2f} puntos")
print(f"R²   (Varianza explicada):   {r2:.2f}")
```

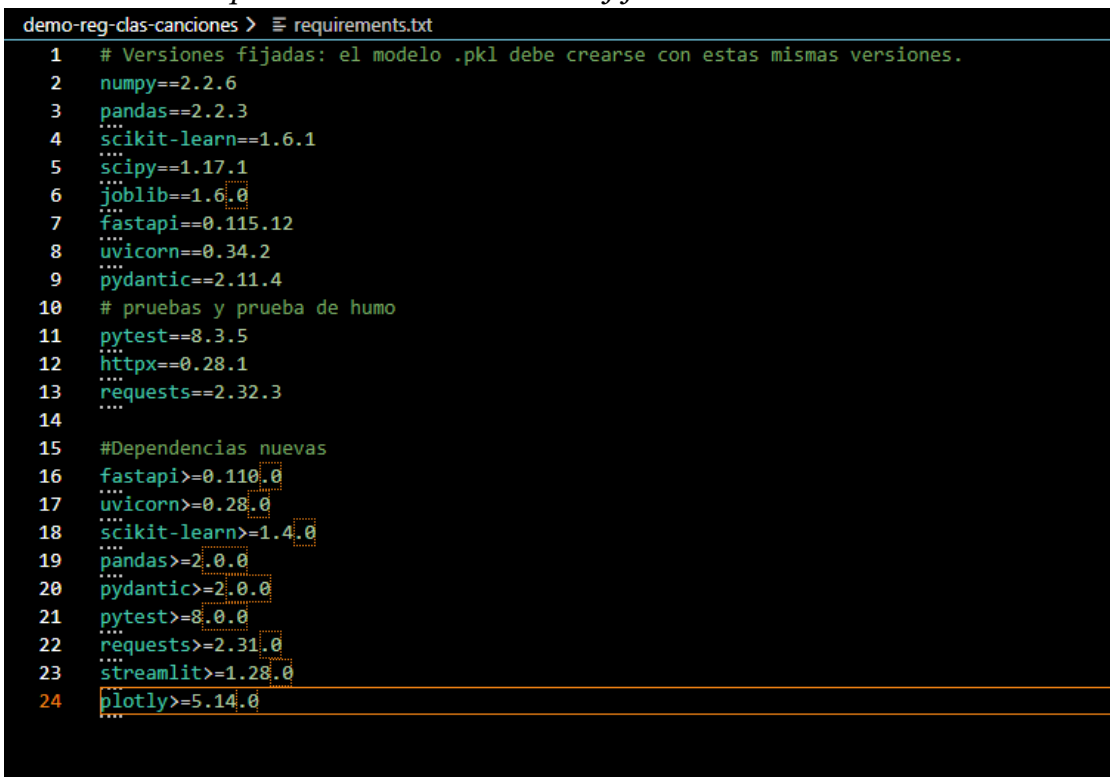
Nota. Celda que ajusta el pipeline con los datos de entrenamiento, predice sobre el conjunto de prueba (2020-2025) y calcula las métricas. La salida reporta MAE de 5,76 puntos, RMSE de 7,15 puntos y R^2 de 0,72. La definición del pipeline no es visible en la captura, por lo que no se precisa qué modelo produjo estos valores; además, difieren de los reportados por la ficha del servicio (Figura 13).

Figura 10. *Ficha del modelo y exportación a modelo.pkl*



Nota. Parte final de la celda de exportación. Se observan la línea base («Promedio histórico»), el desempeño (mae, rmse y r2), la decisión con corte_promocion igual a 50 y el diccionario que empaqueta el pipeline, la ficha y las columnas de entrada. El objeto se guarda en models/modelo.pkl con joblib y la salida confirma «¡Modelo exportado con la ficha validada correctamente!». El corte visible aquí (50) difiere del reportado por el servicio (65) en la Figura 13. La barra del cuaderno indica Python 3.11.9.

Figura 11. *Archivo requirements.txt con versiones fijadas*



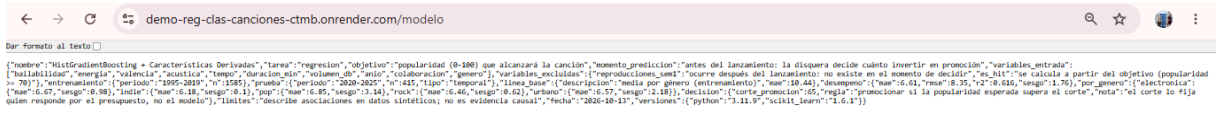
Nota. Contenido de requirements.txt. El comentario inicial señala que el modelo .pkl debe crearse con las mismas versiones fijadas, entre ellas numpy 2.2.6, pandas 2.2.3, scikit-learn 1.6.1, joblib 1.6.0, fastapi 0.115.12, uvicorn 0.34.2 y pydantic 2.11.4, además de pytest, httpx y requests para pruebas. Más abajo aparece un bloque «Dependencias nuevas» con versiones mínimas, que incluye streamlit y plotly. Su relevancia es garantizar la compatibilidad entre el entorno de entrenamiento y el de despliegue.

Figura 12. *Documentación interactiva (Swagger UI) del servicio*



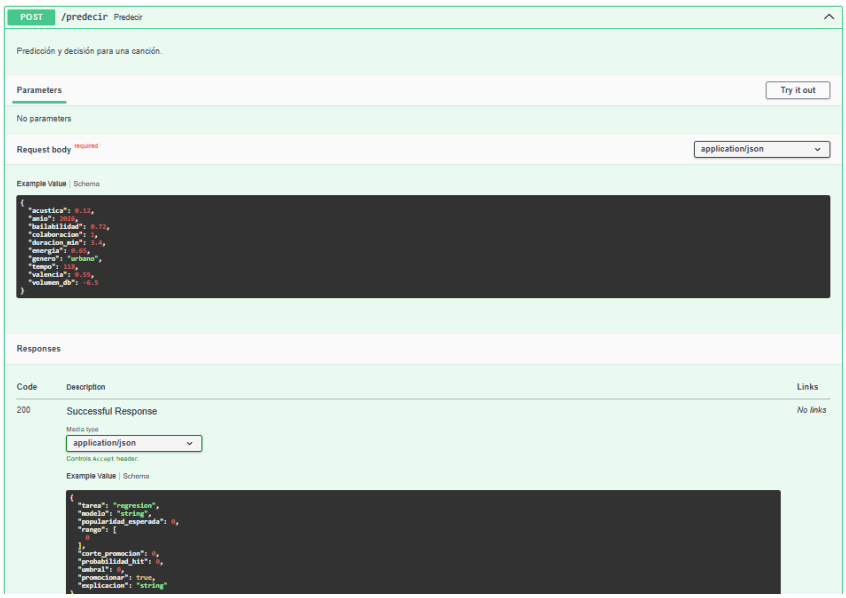
Nota. Página de documentación generada por FastAPI con el título «Predicción de popularidad de canciones», versión 1.0.0 y OpenAPI 3.1. Lista cuatro rutas: GET /salud, GET /modelo, POST /predecir y POST /predecir_lote, y los esquemas Cancion, HTTPValidationError, Prediccion, Salud y ValidationError.

Figura 13. *Ficha del modelo consultada en el servicio desplegado (/modelo)*



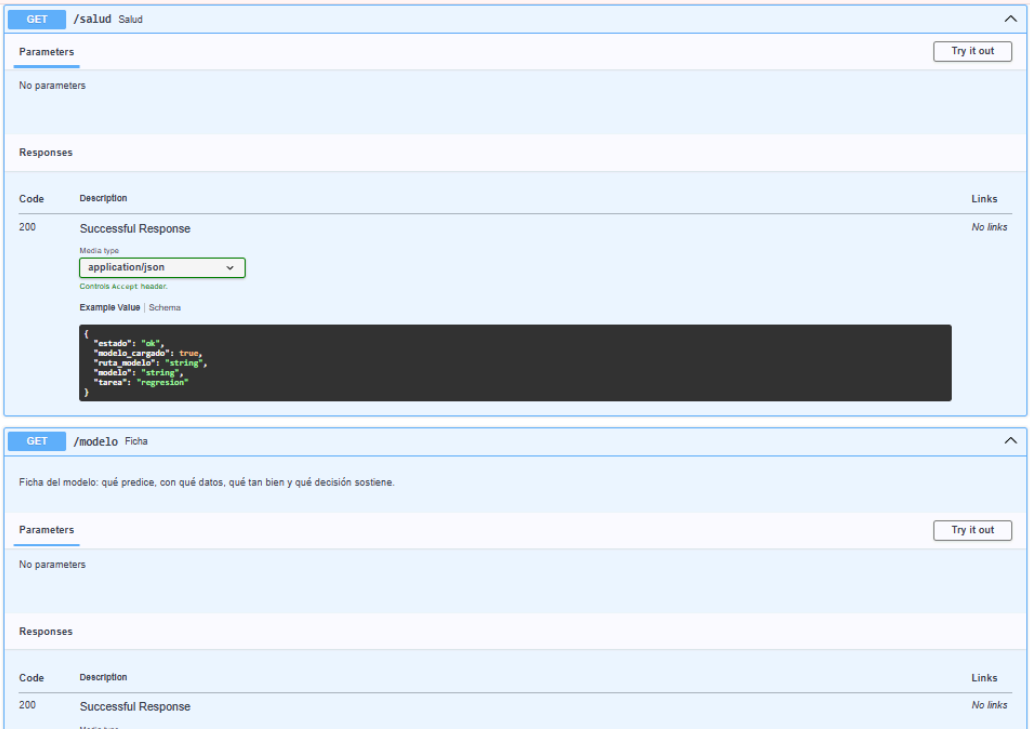
Nota. Respuesta JSON de https://demo-reg-clas-canciones-ctmb.onrender.com/modelo. La ficha indica: modelo HistGradientBoosting con características derivadas, tarea de regresión, objetivo popularidad (0-100) y momento de predicción antes del lanzamiento; variables excluidas reproducciones_sem1 y es_hit; entrenamiento 1995-2019 (n = 1585) y prueba temporal 2020-2025 (n = 415); línea base con MAE de 10,44; desempeño con MAE de 6,61, RMSE de 8,35 y R^2 de 0,616; MAE por género entre 6,18 (indie) y 6,85 (pop); corte de promoción de 65; y la advertencia de que describe asociaciones en datos sintéticos y no es evidencia causal. Registra además Python 3.11.9 y scikit-learn 1.6.1.

Figura 14. *Detalle del endpoint POST /predecir*



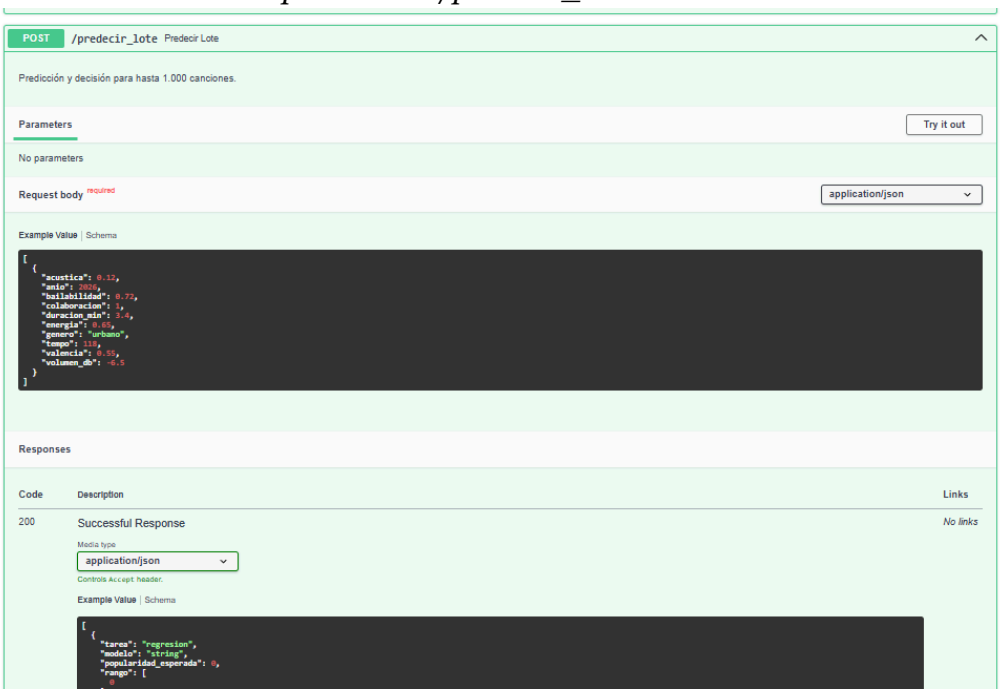
Nota. Sección expandida de POST /predecir, descrita como «Predicción y decisión para una canción». Presenta un ejemplo de cuerpo de solicitud en JSON con las variables de entrada (acustica, anio, bailabilidad, colaboracion, duracion_min, energia, genero, tempo, valencia y volumen_db) y el esquema de la respuesta 200, que incluye tarea, modelo, popularidad_esperada, rango, corte_promocion, probabilidad_hit, umbral, promocionar y explicacion.

Figura 15. *Detalle de los endpoints GET /salud y GET /modelo*



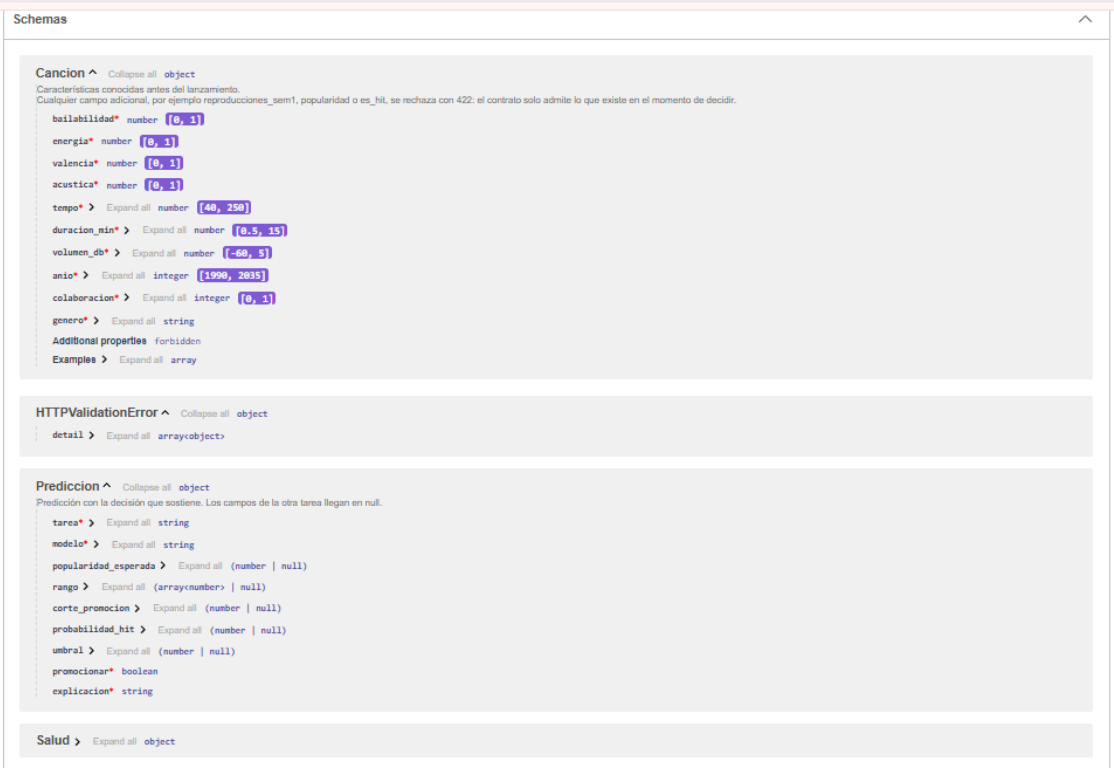
Nota. Secciones expandidas de ambos endpoints, sin parámetros. /salud muestra un ejemplo de respuesta con los campos estado, modelo_cargado, ruta_modelo, modelo y tarea. /modelo se describe como «Ficha del modelo: qué predice, con qué datos, qué tan bien y qué decisión sostiene».

Figura 16. *Detalle del endpoint POST /predecir_lote*



Nota. Sección expandida de POST /predecir_lote, descrita como «Predicción y decisión para hasta 1.000 canciones». El cuerpo de la solicitud es una lista de objetos con las mismas variables de entrada que /predecir, y la respuesta 200 corresponde a una lista de predicciones con la misma estructura.

Figura 17. *Esquemas de datos Cancion y Prediccion*



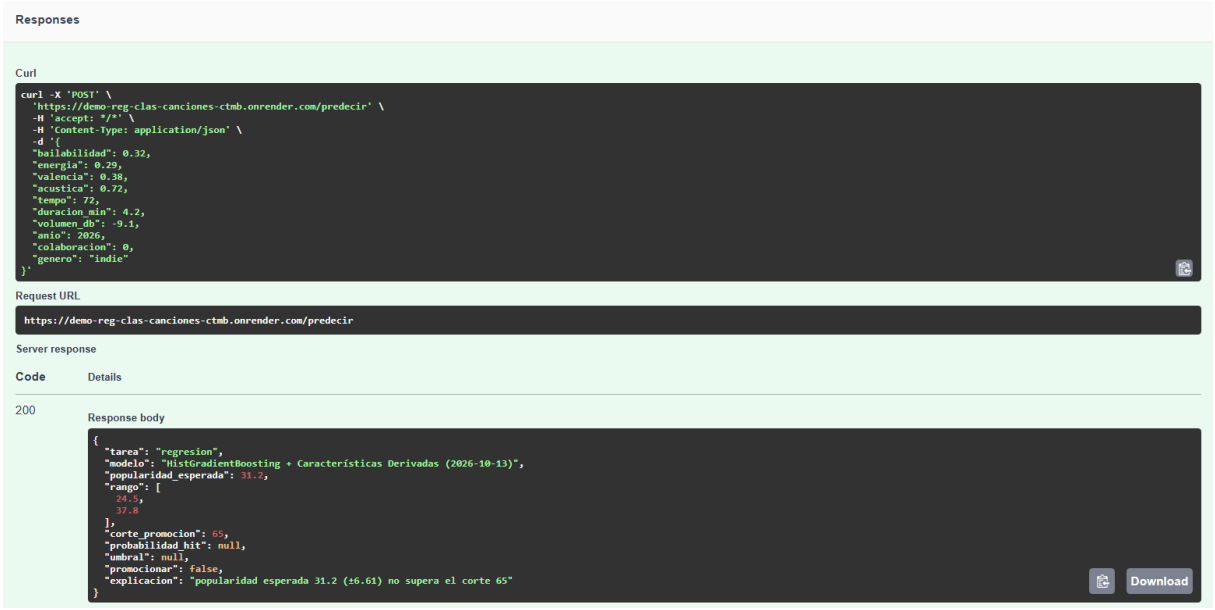
Nota. El esquema Cancion define las características conocidas antes del lanzamiento y sus rangos válidos: bailabilidad, energia, valencia y acustica entre 0 y 1; tempo entre 40 y 250; duracion_min entre 0,5 y 15; volumen_db entre −60 y 5; anio entre 1990 y 2035; colaboracion entre 0 y 1; y genero como texto. Los campos adicionales están prohibidos, y la descripción indica que variables como reproducciones_sem1, popularidad o es_hit se rechazan con error 422. El esquema Prediccion detalla los campos de la respuesta.

Figura 18. *Esquemas de datos Salud y ValidationError*



Nota. Continuación de la sección de esquemas. Salud aparece contraído, por lo que su contenido no es visible en esta captura. ValidationError muestra los campos obligatorios loc, msg y type, utilizados para describir los errores de validación de las solicitudes.

Figura 19. *Ejecución de una predicción en el servicio desplegado*



Nota. Prueba de POST /predecir sobre https://demo-reg-clas-canciones-ctmb.onrender.com/predecir, con una canción de género indie (bailabilidad 0,32, energía 0,29, acústica 0,72, tempo 72, duración 4,2 min, año 2026). El servidor responde con código 200 y devuelve una popularidad esperada de 31,2, con rango de 24,5 a 37,8, corte de promoción de 65 y promocionar igual a false; la explicación indica que 31,2 (±6,61) no supera el corte. Es la evidencia directa de que el servicio publicado recibe datos y entrega predicciones y decisiones.

4. Conclusión

Las evidencias permiten verificar el ciclo completo del miniproyecto: preparación de los datos con partición temporal, entrenamiento y evaluación del modelo, empaquetado en un archivo .pkl, publicación de un servicio con FastAPI y despliegue en Render. El servicio desplegado responde con código 200 en /salud (Figura 3), entrega la ficha del modelo en /modelo (Figura 13) y devuelve predicciones y decisiones en /predecir (Figura 19); además, la prueba automatizada concluye con «servicio en orden» (Figura 1). Según su ficha, el modelo desplegado supera a la línea base (MAE de 6,61 frente a 10,44).